

---

# Sprawozdanie z Laboratorium: Bazy Danych

Konrad Machowski

Jun 25, 2026

## SPIS TREŚCI:

|          |                                                                  |           |
|----------|------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Wstęp do sprawozdania oraz linki</b>                          | <b>2</b>  |
| 1.1      | Wstęp . . . . .                                                  | 2         |
| 1.2      | Wykaz repozytoriów (Linki) . . . . .                             | 2         |
| 1.2.1    | Główne repozytorium sprawozdania (Dokumentacja Sphinx) . . . . . | 2         |
| 1.2.2    | Repozytorium z plikami projektowymi . . . . .                    | 2         |
| 1.2.3    | Repozytoria reszty grupy (Submoduły) . . . . .                   | 2         |
| <b>2</b> | <b>Badania literaturowe</b>                                      | <b>4</b>  |
| <b>3</b> | <b>Dokumentacja modelu bazy danych wypożyczalni samochodów</b>   | <b>5</b>  |
| 3.1      | Wybór zagadnienia i opis procesów . . . . .                      | 5         |
| 3.1.1    | Zagadnienie i jego opis . . . . .                                | 5         |
| 3.1.2    | Opis procesów . . . . .                                          | 5         |
| 3.1.3    | Wykaz gromadzonych danych . . . . .                              | 5         |
| 3.2      | Prototypowy JSON/CSV . . . . .                                   | 5         |
| 3.2.1    | Prototyp CSV . . . . .                                           | 5         |
| 3.2.2    | Prototyp JSON . . . . .                                          | 6         |
| 3.3      | Model koncepcyjny bazy danych . . . . .                          | 6         |
| 3.3.1    | Zidentyfikowane encje . . . . .                                  | 6         |
| 3.3.2    | Zdefiniowane atrybuty . . . . .                                  | 6         |
| 3.3.3    | Opis związków . . . . .                                          | 6         |
| 3.3.4    | Identyfikacja encji słabych . . . . .                            | 6         |
| 3.3.5    | Model w notacji Chena . . . . .                                  | 6         |
| 3.4      | Model logiczny i proces normalizacji . . . . .                   | 6         |
| 3.4.1    | Proces normalizacji . . . . .                                    | 6         |
| 3.4.2    | Ostateczna struktura tabel (3NF) . . . . .                       | 7         |
| 3.4.3    | Schemat ERD w notacji Barkera . . . . .                          | 7         |
| 3.5      | Model fizyczny bazy danych . . . . .                             | 7         |
| 3.5.1    | Model fizyczny dla środowiska SQLite . . . . .                   | 7         |
| 3.5.2    | Model fizyczny dla środowiska PostgreSQL . . . . .               | 7         |
| <b>4</b> | <b>Implementacja bazy danych i import danych</b>                 | <b>8</b>  |
| 4.1      | Wprowadzenie . . . . .                                           | 8         |
| 4.2      | Definicja i inicjalizacja struktur danych . . . . .              | 8         |
| 4.3      | Zasilenie bazy danymi demonstracyjnymi . . . . .                 | 9         |
| 4.4      | Koncepcja mechanizmów masowego importu (ETL) . . . . .           | 9         |
| 4.4.1    | Mechanizm COPY w PostgreSQL . . . . .                            | 9         |
| 4.4.2    | Mechanizm Batch Insert w SQLite . . . . .                        | 10        |
| <b>5</b> | <b>Zapytania do bazy danych</b>                                  | <b>11</b> |
| 5.1      | Zaawansowane mechanizmy zapytań SQL . . . . .                    | 11        |
| 5.1.1    | Selekcja danych i funkcje wierszowe . . . . .                    | 11        |

|       |                                                  |    |
|-------|--------------------------------------------------|----|
| 5.1.2 | Funkcje agregujące i grupowanie . . . . .        | 11 |
| 5.1.3 | Połączenia asymetryczne (LEFT JOIN) . . . . .    | 12 |
| 5.1.4 | Operatory zbiorowe (Różnica) . . . . .           | 12 |
| 5.1.5 | Zagnieżdżone podzapytania (Subqueries) . . . . . | 12 |
| 5.2   | Implementacja skryptowa . . . . .                | 12 |

Niniejsza dokumentacja stanowi kompletne sprawozdanie z laboratorium przedmiotu Bazy Danych. Projekt obejmuje całościową analizę procesów związanych z administracją i projektowaniem relacyjnych baz danych.

Dokumentacja została podzielona na pięć głównych sekcji:

- **Rozdział 1:** Wstęp do sprawozdania oraz wykaz repozytoriów z plikami projektowymi i submodułami.
- **Rozdział 2:** Przegląd źródeł akademickich i naukowo-technicznych dotyczących baz danych, ich architektury, optymalizacji i bezpieczeństwa.
- **Rozdział 3:** Praktyczne modelowanie systemu zarządzania bibliotecznym katalogiem wypożyczeń (analiza wymagań, model konceptualny E-R, model logiczny).
- **Rozdział 4:** Realizacja fizycznego schematu bazy danych poprzez skrypty DDL, mechanizmy importu danych oraz automatyzacja zasilania bazy w środowiskach PostgreSQL i SQLite.
- **Rozdział 5:** Zaawansowane funkcje SQL w Pythonie, selekcje, agregacje, złączenia, operatory zbiorowe i podzapytania.

## WSTĘP DO SPRAWOZDANIA ORAZ LINKI

### Autorzy

1. Olaf Chomicki
2. Konrad Machowski
3. Wiktor Wydrzyński

## 1.1 Wstęp

Niniejsze sprawozdanie stanowi kompleksową dokumentację prac laboratoryjnych zrealizowanych w ramach przedmiotu Bazy Danych. Głównym założeniem projektu było przełożenie wiedzy akademickiej na praktyczne umiejętności inżynierskie, niezbędne przy projektowaniu, implementacji oraz utrzymaniu nowoczesnych systemów relacyjnych baz danych.

Opracowanie przeprowadza czytelnika przez pełen cykl życia systemu bazodanowego. Proces rozpoczął się od dogłębnej analizy wymagań teoretycznych, co posłużyło jako fundament do zaprojektowania modelu pojęciowego i logicznego. Następnie architektura ta została powołana do życia poprzez fizyczną implementację w środowiskach PostgreSQL oraz SQLite. Istotnym elementem projektu była również automatyzacja i realizacja procesów wsadowego zasilania struktury rzeczywistymi danymi.

## 1.2 Wykaz repozytoriów (Linki)

Zgodnie z dobrymi praktykami inżynierii oprogramowania, projekt został logicznie rozdzielony na odrębne repozytoria w rozproszonym systemie kontroli wersji Git. Takie podejście gwarantuje utrzymanie porządku, separację dokumentacji tekstowej od kodu wykonywalnego oraz ułatwia śledzenie historii zmian w poszczególnych komponentach.

### 1.2.1 Główne repozytorium sprawozdania (Dokumentacja Sphinx)

Niniejsze repozytorium pełni rolę centralnego węzła dokumentacyjnego. Znajduje się w nim pełna struktura tekstowa zapisana w formacie reStructuredText, niezbędna konfiguracja generatora dokumentacji Sphinx (plik `conf.py`) oraz pliki konfiguracyjne środowiska budowania (np. pliki `Makefile`).

- **Link:** <https://github.com/KMachoK/Repozytorium-Glowne.git>

### 1.2.2 Repozytorium z plikami projektowymi

To repozytorium przechowuje właściwe pliki inżynierskie i deweloperskie. Zgromadzono w nim skrypty DDL definiujące struktury dla silników PostgreSQL i SQLite, wygenerowany plik lokalnej bazy danych oraz zbiory danych w formacie CSV, które wykorzystano do automatycznego zasilenia poszczególnych tabel.

- **Link:** <https://github.com/KMachoK/Repozytorium-Glowne.git>

### 1.2.3 Repozytoria reszty grupy (Submoduły)

Poniższa sekcja zawiera wykaz repozytoriów, w których zespoły opracowywały dedykowane tematy specjalistyczne. Aby zapewnić spójność ostatecznego dokumentu, repozytoria te zostały zintegrowane z głównym projektem jako submoduły (submodules) i stanowią treść Rozdziału 2.

- **Grupa 1 (rozdział\_1):** <https://github.com/karaskamil/Sprzet-dla-bazy-danych.git>
- **Grupa 2 (rozdział\_2):** [https://github.com/Youarecheck/Bazy\\_Danych\\_Tematyczne\\_Repo\\_MK.git](https://github.com/Youarecheck/Bazy_Danych_Tematyczne_Repo_MK.git)

- **Grupa 3 (rozdzial\_3):** <https://github.com/pawlos1337/Bazy-danych-temat.git>
- **Grupa 4 (rozdzial\_4):** [https://github.com/OskarProgrammer/monitorowanie\\_i\\_diagnostyka.git](https://github.com/OskarProgrammer/monitorowanie_i_diagnostyka.git)
- **Grupa 5 (rozdzial\_5):** <https://github.com/KMachoK/Tematyczne.git>
- **Grupa 6 (rozdzial\_6):** <https://github.com/domino0472/Partycjonowani-Danych>
- **Grupa 7 (rozdzial\_7):** <https://github.com/oski486/BazyDanych-Subject.git>
- **Grupa 8 (rozdzial\_8):** <https://github.com/Koko9077/Kopie-zapasowe-i-odzyskiwanie-danych.git>

## BADANIA LITERATUROWE

## **DOKUMENTACJA MODELU BAZY DANYCH WYPOŻYCZALNI SAMOCHODÓW**

### **Autorzy**

1. Olaf Chomicki
2. Konrad Machowski
3. Wiktor Wydrzyński

## **3.1 Wybór zagadnienia i opis procesów**

### **3.1.1 Zagadnienie i jego opis**

System zarządzania flotą i wynajmem w wypożyczalni samochodów. Projekt obejmuje ewidencję klientów, dostępnych pojazdów z podziałem na kategorie cenowe, a także pełen proces wypożyczeń aut przez klientów, z określeniem czasu wynajmu i statusu operacji.

### **3.1.2 Opis procesów**

Głównym procesem w systemie jest obieg samochodu między wypożyczalnią a klientem.

- **Rejestracja klienta:** Klient jest dodawany do bazy, system wymaga podania danych osobowych i kontaktowych. Wymagana jest unikalność numeru PESEL oraz Numeru Prawa Jazdy.
- **Katalogowanie floty:** Samochody posiadają unikalne numery rejestracyjne i są przypisywane do grup cenowych (Kategorii).
- **Wypożyczenie:** Rejestracja faktu wynajmu auta, który łączy konkretnego klienta z pojazdem w zdefiniowanym przedziale czasu (od-do). Proces ten otrzymuje status (np. Zakończone, W\_trakcie, Zarezerwowane).
- **Więzy integralności:** Usunięcie lub modyfikacja samochodu/klienta powiązanego z rezerwacją musi być kontrolowana przez system (np. poprzez klucze obce), aby zapobiegać niespójności bazy.

### **3.1.3 Wykaz gromadzonych danych**

- **Dane klienta:** Imie, Nazwisko, PESEL, Nr\_Prawa\_Jazdy, Telefon, Email.
- **Dane auta:** Marka, Model, Rok\_Produkcji, Nr\_Rejestracyjny.
- **Dane kategorii:** Nazwa kategorii, Cena za dzień.
- **Dane transakcyjne:** Data\_Od, Data\_Do, Status wypożyczenia.

## **3.2 Prototypowy JSON/CSV**

Aby zweryfikować kompletność wprowadzanych danych, przygotowano pliki zawierające próbkowe dane.

### **3.2.1 Prototyp CSV**

```
Imie,Nazwisko,PESEL,Nr_Prawa_Jazdy,Telefon,Email  
Piotr,Wiśniewski,59081618205,84679/96/1949,+48706147330,piotr.wiśniewski8@email.com
```



### 3.2.2 Prototyp JSON

```
{
  "klienci": [
    {
      "Imie": "Piotr",
      "Nazwisko": "Wiśniewski",
      "PESEL": "59081618205",
      "Nr_Prawa_Jazdy": "84679/96/1949",
      "Telefon": "+48706147330",
      "Email": "piotr.wisniewski8@email.com"
    }
  ]
}
```

## 3.3 Model koncepcyjny bazy danych

### 3.3.1 Zidentyfikowane encje

- Klient
- Samochód
- Kategoria
- Wypożyczenie

### 3.3.2 Zdefiniowane atrybuty

- **Klient:** Imie, Nazwisko, PESEL, Nr\_Prawa\_Jazdy, Telefon, Email
- **Samochód:** Marka, Model, Rok\_Produkcji, Nr\_Rejestracyjny
- **Kategoria:** Nazwa, Cena\_Za\_Dzien
- **Wypożyczenie:** Data\_Od, Data\_Do, Status

### 3.3.3 Opis związków

- **Kategoria – Samochód (1:N):** Do jednej kategorii może być przypisanych wiele samochodów, ale samochód należy tylko do jednej kategorii.
- **Klient – Wypożyczenie (1:N):** Pojedynczy klient ma możliwość wypożyczania aut wielokrotnie.
- **Samochód – Wypożyczenie (1:N):** Samochód może być w historii swojej eksploatacji wypożyczony przez wielu różnych klientów.

### 3.3.4 Identyfikacja encji słabych

Ze względu na występowanie relacji wiele-do-wielu (M:N) pomiędzy encjami Klient i Samochód, utworzono encję asocjacyjną (słabą) o nazwie **Wypożyczenia**. Rozwiązuje ona pułapkę połączeń (wiatrak) i ewidencjonuje każdą transakcję.

### 3.3.5 Model w notacji Chena

## 3.4 Model logiczny i proces normalizacji

### 3.4.1 Proces normalizacji

Zastosowano proces normalizacji, doprowadzając relacje do 3 Postaci Normalnej (3NF). Każda tabela posiada klucz główny (PK). Aby uniknąć redundancji i anomalii modyfikacji, atrybuty określające stawkę cenową przeniesiono do słownikowej tabeli Kategorie. Utworzono relacyjne klucze obce (FK), gwarantujące integralność referencyjną.

### 3.4.2 Ostateczna struktura tabel (3NF)

- **Kategorie:** ID\_Kategorii (PK), Nazwa, Cena\_Za\_Dzien
- **Klienci:** ID\_Klienta (PK), Imie, Nazwisko, PESEL, Nr\_Prawa\_Jazdy, Telefon, Email
- **Samochody:** ID\_Samochodu (PK), ID\_Kategorii (FK), Marka, Model, Nr\_Rejestracyjny, Rok\_Produkcji
- **Wypożyczenia:** ID\_Wypozyczenia (PK), ID\_Klienta (FK), ID\_Samochodu (FK), Data\_Od, Data\_Do, Status

### 3.4.3 Schemat ERD w notacji Barkera

## 3.5 Model fizyczny bazy danych

### 3.5.1 Model fizyczny dla środowiska SQLite

SQLite korzysta z podstawowych typów TEXT, INTEGER i REAL. Brak natywnych typów daty wymaga zapisywania ich jako ciągów znaków (TEXT). Do kluczy głównych zastosowano automatyczną sekwencję INTEGER PRIMARY KEY.

- **Kategorie:** ID\_Kategorii : INTEGER PRIMARY KEY, Nazwa : TEXT, Cena\_Za\_Dzien : REAL
- **Klienci:** ID\_Klienta : INTEGER PRIMARY KEY, Imie : TEXT, Nazwisko : TEXT, PESEL : TEXT UNIQUE, Nr\_Prawa\_Jazdy : TEXT UNIQUE, Telefon : TEXT, Email : TEXT
- **Samochody:** ID\_Samochodu : INTEGER PRIMARY KEY, Marka : TEXT, Model : TEXT, Nr\_Rejestracyjny : TEXT UNIQUE, Rok\_Produkcji : INTEGER, ID\_Kategorii : INTEGER REFERENCES Kategorie
- **Wypożyczenia:** ID\_Wypozyczenia : INTEGER PRIMARY KEY, ID\_Klienta : INTEGER REFERENCES Klienci, ID\_Samochodu : INTEGER REFERENCES Samochody, Data\_Od : TEXT, Data\_Do : TEXT, Status : TEXT

### 3.5.2 Model fizyczny dla środowiska PostgreSQL

Baza PostgreSQL umożliwiła implementację ścisłych typów natywnych, optymalizujących miejsce i wydajność silnika z wykorzystaniem VARCHAR, DATE i formatowania liczbowego NUMERIC. Zastosowano typ SERIAL do obsługi sekwencji kluczy głównych.

- **Kategorie:** ID\_Kategorii : SERIAL PRIMARY KEY, Nazwa : VARCHAR(50), Cena\_Za\_Dzien : NUMERIC(10,2)
- **Klienci:** ID\_Klienta : SERIAL PRIMARY KEY, Imie : VARCHAR(50), Nazwisko : VARCHAR(50), PESEL : VARCHAR(11) UNIQUE, Nr\_Prawa\_Jazdy : VARCHAR(20) UNIQUE, Telefon : VARCHAR(15), Email : VARCHAR(100)
- **Samochody:** ID\_Samochodu : SERIAL PRIMARY KEY, Marka : VARCHAR(50), Model : VARCHAR(50), Nr\_Rejestracyjny : VARCHAR(20) UNIQUE, Rok\_Produkcji : INT, ID\_Kategorii : INT REFERENCES Kategorie
- **Wypożyczenia:** ID\_Wypozyczenia : SERIAL PRIMARY KEY, ID\_Klienta : INT REFERENCES Klienci, ID\_Samochodu : INT REFERENCES Samochody, Data\_Od : DATE, Data\_Do : DATE, Status : VARCHAR(20)

## IMPLEMENTACJA BAZY DANYCH I IMPORT DANYCH

### 4.1 Wprowadzenie

W ramach czwartego rozdziału zrealizowano dwa kluczowe etapy prac wdrożeniowych: utworzenie struktur tabelarycznych zdefiniowanych w modelu fizycznym oraz wdrożenie zoptymalizowanych mechanizmów importu danych testowych. Prace przeprowadzono w sposób równoległy dla środowiska lokalnego opartego na silniku SQLite oraz serwerowego wykorzystującego system PostgreSQL.

### 4.2 Definicja i inicjalizacja struktur danych

Pierwszym etapem wdrażania systemu bazodanowego było przeniesienie założeń modelu fizycznego do wytypowanych systemów zarządzania bazami danych (DBMS). Proces ten wymagał adaptacji skryptów DDL (Data Definition Language) do specyficznych dialektów SQL obsługiwanych przez wybrane środowiska.

W przypadku **PostgreSQL**, definicje tabel utworzono wykorzystując zaawansowane typy danych (np. NUMERIC) oraz współczesne metody autoinkrementacji kluczy głównych poprzez klauzulę **GENERATED ALWAYS AS IDENTITY**. Poniżej przedstawiono implementację dla kluczowych tabel, ilustrującą nałożone więzy integralności (unikalność i kontrolę dat):

```
-- Fragment definicji kluczowych tabel (PostgreSQL)
CREATE TABLE Klienci (
    ID_Klienta INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    Imie VARCHAR(50) NOT NULL,
    Nazwisko VARCHAR(100) NOT NULL,
    PESEL VARCHAR(11) UNIQUE NOT NULL,
    Nr_Prawa_Jazdy VARCHAR(20) UNIQUE
);

CREATE TABLE Wypozyczenia (
    ID_Wypozyczenia INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    ID_Klienta INT NOT NULL,
    Data_Od DATE NOT NULL,
    Data_Do DATE NOT NULL,
    CONSTRAINT fk_klient FOREIGN KEY (ID_Klienta) REFERENCES Klienci(ID_Klienta),
    CONSTRAINT sprawdz_daty CHECK (Data_Do >= Data_Od)
);
```

W środowisku **SQLite**, będącym lekką bazą wbudowaną, wykorzystano uproszczone mapowanie typów (INTEGER, TEXT). Proces inicjalizacji wymagał wywołania procedury aktywującej wsparcie dla więzów referencyjnych za pomocą dyrektywy **PRAGMA foreign\_keys = ON**;, co zostało zaprezentowane na poniższym fragmencie kodu implementującego tabele z kluczem obcym:

```
-- Fragment definicji kluczowych tabel (SQLite)
PRAGMA foreign_keys = ON;

CREATE TABLE Klienci (
    ID_Klienta INTEGER PRIMARY KEY,
```

(continues on next page)

(continued from previous page)

```

    Imie TEXT NOT NULL,
    Nazwisko TEXT NOT NULL,
    PESEL TEXT UNIQUE NOT NULL
);

CREATE TABLE Wypozyczenia (
    ID_Wypozyczenia INTEGER PRIMARY KEY,
    ID_Klienta INTEGER,
    Data_Od TEXT NOT NULL,
    Data_Do TEXT NOT NULL,
    FOREIGN KEY (ID_Klienta) REFERENCES Klienci(ID_Klienta)
);

```

### 4.3 Zasilenie bazy danymi demonstracyjnymi

Na etapie testowania architektury modelu fizycznego, puste relacje zostały zasilone małym zbiorem danych demonstracyjnych wykorzystując klasyczne polecenia DML z rodziny INSERT. Procedura ta miała na celu weryfikację poprawności działania nałożonych ograniczeń integralnościowych – na przykład weryfikację unikalności numerów PESEL i numerów rejestracyjnych pojazdów oraz zgodność typów asocjacyjnych.

### 4.4 Koncepcja mechanizmów masowego importu (ETL)

Dla zasymulowania warunków produkcyjnych i obciążenia systemów większym wolumenem rekordów, utworzono standaryzowany plik formatu CSV reprezentujący logikę płaskich rekordów z informacjami o potencjalnych klientach.

Zarówno dla SQLite, jak i PostgreSQL, pierwszym krokiem w środowisku języka Python było odczytanie płaskiego pliku i przeniesienie go do ustrukturyzowanej, iterowalnej pamięci RAM (np. w postaci listy list):

```

import csv

# Wczytywanie danych z pliku CSV do struktury iterowalnej
with open('klienci.csv', 'r', encoding='utf-8') as plik_csv:
    reader = csv.reader(plik_csv)
    next(reader) # Pomińcie wiersza z nagłówkami
    dane_z_csv = [row for row in reader]

```

Mając przygotowaną strukturę w pamięci aplikacji, przystąpiono do operacji ładowania danych, przyjmując strategię adekwatną do natury silników bazodanowych.

#### 4.4.1 Mechanizm COPY w PostgreSQL

Ze względu na architekturę klient-serwer, przesyłanie tysięcy pojedynczych zapytań INSERT wiąże się z ogromnym kosztem operacyjnym ze względu na opóźnienia sieciowe oraz narzut parsowania po stronie serwera. Wdrożono zatem natywną dla PostgreSQL instrukcję COPY obsługującą strumieniowanie wejścia (STDIN):

```

# Właściwy proces wsadowego importu do bazy PostgreSQL
# (Zakładając aktywne połączenie psycopg2/psycopg3 pod zmienną conn_pg)

with conn_pg.cursor() as cursor_pg:
    # Zastosowanie strumieniowania binarnego wprost do pamięci buforowej Postgresa
    with cursor_pg.copy(
        "COPY Klienci (Imie, Nazwisko, PESEL, Nr_Prawa_Jazdy, Telefon, Email) FROM_
↪STDIN"

```

(continues on next page)

(continued from previous page)

```
) as copy_operation:
    for row in dane_z_csv:
        copy_operation.write_row(row)

conn_pg.commit()
```

Takie podejście buduje ciągły potok danych bezpośrednio do pliku tabeli, omijając standardową kontrolę transakcyjną planisty (query planner) dla każdego pojedynczego wiersza z osobna, co jest referencyjnym wzorcem w środowiskach inżynierii danych.

#### 4.4.2 Mechanizm Batch Insert w SQLite

W silniku SQLite wykorzystano z kolei grupowanie instrukcji poprzez metodę `executemany()` dostępną w sterowniku języka Python. Ponieważ cała baza jest jednoplikowa, wykonanie całego wsadu w obrębie jednej otwartej i zatwierdzonej tylko raz na końcu transakcji redukuje obciążenie I/O systemu plików do minimum.

## **ZAPYTANIA DO BAZY DANYCH**

W ramach piątego rozdziału zaimplementowano moduł analityczny w języku Python, służący do interakcji ze strukturami relacyjnej bazy danych. Przygotowane zapytania SQL realizują złożone scenariusze biznesowe wypożyczalni samochodów, wykorzystując zaawansowane mechanizmy silnika bazy danych.

Zgodnie z dobrymi praktykami inżynierii oprogramowania, kwerendy zhermetyzowano w postaci funkcji przyjmujących aktywny wskaźnik połączenia (obiekt `conn`), co pozwala na bezpośrednie użycie kodu zarówno w środowiskach produkcyjnych, jak i w notatnikach JupyterLab (JupyterHub).

### **5.1 Zaawansowane mechanizmy zapytań SQL**

Aby system w pełni odpowiadał na potrzeby analityczne, w kodzie Pythona wdrożono pełne spektrum operacji na relacjach, w tym selekcję z operacjami wierszowymi, agregacje wielowierszowe, algebrę zbiorów, złączenia asymetryczne oraz podzapytania skalarne. Poniżej omówiono zaimplementowaną logikę bazodanową.

#### **5.1.1 Selekcja danych i funkcje wierszowe**

Pierwsze z zapytań wykorzystuje klauzulę `WHERE` do filtrowania rekordów po statusie transakcji oraz implementuje wierszowe operacje arytmetyczne na typach dat (natywnie zwracające typ `integer` w PostgreSQL).

**pobierz\_czas\_wypozycczen**(*conn*)

Pobiera wykaz zakończonych wypożyczeń wraz z dynamicznie obliczanym czasem ich trwania.

**Parameters**

**conn** – Aktywny obiekt połączenia z bazą danych (np. `psycopg`).

**Returns**

Lista krotek: (ID wypożyczenia, złączona nazwa auta, wyliczona liczba dni).

**Opis techniczny:** Zapytanie realizuje operację arytmetyczną (`w.data_do - w.data_od`) bezpośrednio na warstwie bazy danych dla rekordów ze statusem 'Zakończony'. Dodatkowo, operator `||` pozwala na wierszową konkatencję atrybutów marki i modelu do jednej, płaskiej kolumny.

#### **5.1.2 Funkcje agregujące i grupowanie**

Do generowania metryk biznesowych wykorzystano funkcje agregujące, operujące na zgrupowanych wierszach dla poszczególnych osi wymiarów.

**raport\_przychodow\_kategorii**(*conn*)

Generuje zsumowany raport estymowanych przychodów dla klasyfikacji pojazdów.

**Parameters**

**conn** – Aktywny obiekt połączenia z bazą danych.

**Returns**

Lista krotek: (Nazwa kategorii, liczba wypożyczeń, łączny przychód).

**Opis techniczny:** Kwerenda implementuje wielokrotne złączenie wewnętrzne (`INNER JOIN`) celem utworzenia widoku o płaskiej hierarchii. Wynik jest grupowany po nazwie kategorii (`GROUP BY k.nazwa`), a przychód estymowany z wykorzystaniem matematycznego wyrażenia wewnątrz funkcji grupującej `SUM()`.

### 5.1.3 Połączenia asymetryczne (LEFT JOIN)

W relacyjnych systemach analitycznych niezbędne bywa zachowanie informacji o encjach sierocych (nieposiadających powiązań w tabelach zależnych).

**sprawdz\_historie\_aut**(*conn*)

Zwraca pełny wykaz pojazdów we flocie, identyfikując jednostki z brakiem historii operacyjnej.

**Parameters**

**conn** – Aktywny obiekt połączenia z bazą danych.

**Returns**

Lista krotek z danymi pojazdu oraz opcjonalną datą wypożyczenia.

**Opis techniczny:** Zastosowanie klauzuli LEFT JOIN między tabelą główną (samochody) a zależną (wypożyczenia) gwarantuje, że pojazdy niespełniające warunku złączenia zostaną zwrócone w relacji wynikowej z wartością NULL w miejscu atrybutów tabeli podrzędnej.

### 5.1.4 Operatory zbiorowe (Różnica)

Część logiki oparto o matematyczną algebrę relacyjną zamiast tradycyjnych złączeń warunkowych, co ułatwia zarządzanie inwentarzem w czasie rzeczywistym.

**dostepne\_samochody**(*conn*)

Zwraca zbiór identyfikatorów pojazdów aktualnie dostępnych fizycznie na placu.

**Parameters**

**conn** – Aktywny obiekt połączenia z bazą danych.

**Returns**

Lista krotek identyfikujących dostępne samochody.

**Opis techniczny:** W zapytaniu użyto operatora różnicy zbiorów EXCEPT. System zaciąga nadzbiór wszystkich zarejestrowanych aut, a następnie odejmuje od niego podzbiór generowany przez kwerendę wskazującą pojazdy mające w tabeli transakcyjnej status 'W trakcie'.

### 5.1.5 Zagnieżdżone podzapytania (Subqueries)

Logikę najdroższego zasobu odseparowano strukturalnie, wstrzykując zapytanie podrzędne bezpośrednio do predykatu kwerendy nadrzędnej.

**klienci\_najdrozszych\_aut**(*conn*)

Wyciąga dane kontaktowe klientów korzystających z najdroższego segmentu floty.

**Parameters**

**conn** – Aktywny obiekt połączenia z bazą danych.

**Returns**

Zbiór krotek (imię, nazwisko, email, telefon).

**Opis techniczny:** Rdzeniem zapytania jest dynamiczne podzapytanie w klauzuli WHERE określające maksymalną stawkę bazową: (SELECT id\_kategorii FROM kategorie ORDER BY cena\_za\_dzien DESC LIMIT 1). Zapewnia to odporność aplikacji na zmiany cenifikatora w tabeli kategorii.

## 5.2 Implementacja skryptowa

Wyżej zdefiniowane funkcje zostały spakietowane w postaci odizolowanego modułu Pythona, gotowego do wykonania lub podpięcia jako biblioteka analityczna w głównym kodzie aplikacji.

```
import psychopg
import sqlite3

def sprawdz_historie_aut(conn):
    cursor = conn.cursor()
    query = """
    SELECT
```

(continues on next page)

(continued from previous page)

```
s.marka,  
s.model,  
s.nr_rejestracyjny,  
w.data_od  
FROM samochody s  
LEFT JOIN wypozyczenia w ON s.id_samochodu = w.id_samochodu;  
""""  
cursor.execute(query)  
wyniki = cursor.fetchall()  
cursor.close()  
return wyniki
```